

Stage 0 Step-by-Step Practical & Conceptual Recap

1. Directory Pathing & Execution Scope

- **What Happened:** When you initially ran `node server.js`, Node threw a `MODULE_NOT_FOUND` error because your terminal prompt was sitting in your home root (`C:\Users\lernes`), where `server.js` did not exist.
- **The Fix:** We navigated to your specific nested path using quotes to handle spaces in folder names:

```
cd "C:\Users\lernes\Documents\Ernest Career\Training\FlyRank Internship\Week2\BackEnd Engineering\Build your first CRUD API\NodeJS"
```
- **Sysadmin Analogy:** In Linux/Windows administration, executables only search the Current Working Directory (`.`) or defined system binaries in `PATH`. Running a script without entering its workspace folder is like trying to execute a system daemon script from `/var/log/` when the file lives in `/opt/app/`.

2. File Naming & Entry Point Mismatches

- **What Happened:** VS Code initially had the file saved as `index.cjs`, but the terminal command requested `node server.js`.
- **The Fix:** Renaming `index.cjs` to `server.js` synchronized the runtime execution command with the physical file on disk.
- **Sysadmin Analogy:** This is identical to configuring a systemd service unit file (`/etc/systemd/system/myapp.service`) to point to `/usr/bin/python main.py`, but having the actual script named `app.py`. The daemon manager fails because the target binary module name is incorrect.

3. Foreground Processes vs Multi-Tab Workflows

- **What Happened:** After starting `node server.js`, the server output `Example app listening on port 3000` and froze the terminal prompt.
- **The Fix:** Instead of killing the server (`Ctrl + C`), we opened a secondary terminal tab in VS Code.
- **Sysadmin Analogy:** Web servers run as persistent background daemons or foreground services. If you start a service interactively without detaching it (or without `systemd/tmux` backgrounding), it retains control of standard input/output (`stdin/stdout`). Opening a second shell tab simulates opening an SSH session to send client traffic while the daemon continues running on the server host.

4. HTTP Inspection & Raw Protocol Verification (`curl -i`)

- **What Happened:** We executed `curl -i http://localhost:3000/` from the second terminal prompt.
- **The Response:**
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 12

Hello World!

- **Sysadmin Analogy:** The `-i` flag forces curl to display the raw HTTP response headers along with the status code before printing the payload body. In network engineering and API compliance, checking status codes (200 OK) and content headers (Content-Type) is how we verify routing rules and payload formats prior to frontend integration.

5. Environment Isolation & Local Git Identity Scoping

- **What Happened:** You needed your commits attributed to your FlyRank email (ernest.mwangombe@10x.ai) rather than your personal Gmail linked to VS Code.
- **The Fix:** We omitted the `--global` flag when configuring Git identity inside the repo folder:
`git config user.name "ernestmwangombe"`
`git config user.email "ernest.mwangombe@10x.ai"`
- **Sysadmin Analogy:** This is the difference between setting global system environment variables (`/etc/environment`) versus scoping configuration directives locally inside a service file (`/etc/nginx/conf.d/app.conf`). Scoped configuration ensures your internship repository identity remains isolated without affecting your personal global development profile.

6. Repository Hygiene (.gitignore & Package Bloat)

- **What Happened:** Running `git add .` initially staged thousands of third-party dependency files inside `node_modules/`.
- **The Fix:** We staged a `.gitignore` file and detached the dependencies from Git tracking:
`git rm -r --cached node_modules`
- **Sysadmin Analogy:** Committing `node_modules/` to Git is like backing up volatile cache folders (`/tmp/` or `/var/cache/`) into a system configuration backup repository. Dependencies belong in `package.json` so they can be re-installed dynamically using `npm install` on any server host.

eof

This detailed step-by-step connection gives you a solid foundation mapping real terminal steps to core architecture concepts.

7. Git Repository Inspection & Audit Commands

- **What Happened:** You wanted a way to review saved commits and verify repository status.
- **The Fix & Commands:**
 - **git log --oneline:** Shows the condensed timeline of saved commit hashes and messages.
 - **git status:** Shows untracked, modified, or staged files in the working directory.
 - **git show:** Displays the exact line-by-line diff of the most recent commit.
- **Sysadmin Analogy:** Running **git log** is like reviewing `/var/log/syslog` or system audit logs to track changes made over time. Running **git status** functions like **systemctl status** to inspect the live health and state of your workspace before applying new configuration updates.